FastBPE: Benchmarking and Optimizing Tokenizer Spee

Research-style benchmark report

FastBPE benchmarks tokenizer throughput, latency, memory, batching behavior, and domain sensitivity across English, code, web, and technical text. In the current five-trial run on Windows-10-10.0.26200-SP0, tiktoken was the strongest average MB/s performer at 4.610 MB/s. SentencePiece remained competitive and led the code domain at 4.154 MB/s. The cached Python tokenizer improved on the naive Python baseline by 1.49x in MB/s with higher memory usage.

Hypotheses

H1. Production native tokenizer libraries will outperform readable Python baselines on throughput and latency.

H2. Tokenizer rankings will change across English prose, code, noisy web text, and technical markdown-like text.

H3. Repeated-substring caching will improve the custom Python baseline while increasing memory usage.

H4. Batch encoding will favor tokenizers that expose efficient multi-document paths, but speed gains will not be uniform.

H5. Exact token-ID compatibility cannot be claimed unless the tokenizer intentionally matches the reference vocabulary and merge logic.

Methodology and Experimental Design

Environment. Platform: Windows-10-10.0.26200-SP0. Python: 3.11.9. Processor: Intel64 Family 6 Model 189 Stepping 1, GenuineIntel.

Tokenizers. tiktoken, hf, sentencepiece, naive, cached.

Domains. code, english, technical, web.

Controls. Warmup before timing, separate cold-start measurement, identical document sets per run, five trials, and raw CSV/JSON output.

Datasets. This run used persisted synthetic corpora in data/ rather than the in-memory homogeneous fallback. The corpus was diversified specifically to reduce misleading cache behavior from repeated identical documents.

Exactness. tiktoken is the reference tokenizer, but this run did not include a non-reference tokenizer that claimed token-ID compatibility, so exact-match tables are absent by design.

Experiments

Experiment 1: Overall tokenizer speed, measured in MB/s, tokens/s, documents/s, and latency percentiles.

Experiment 2: Domain sensitivity across clean English, code, noisy web text, and technical text.

Experiment 3: Input-length scaling across the available length buckets in the current corpus.

Experiment 4: Batch versus single encoding, reported for tokenizers that support encode_batch.

Experiment 5: Exactness testing, included as a framework feature but not exercised for a non-reference compatible tokenizer in this run.

Experiment 6: Caching optimization, comparing the naive and cached Python prototypes.

Overall Results by Tokenizer

This table aggregates mean performance across all domains and trials.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	4.610	1083855	0.0378	4190.0
sentencepiece	3.764	1508537	0.0445	4858.8
hf	3.070	731070	0.0562	7906.0
cached	2.001	1794249	0.0927	8390.2
naive	1.342	1201147	0.1291	6462.9

Top Tokenizer-Domain Configurations

These are the strongest individual tokenizer-domain combinations by MB/s.

tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	english	5.680	970210	0.0293	3671.2
tiktoken	web	4.562	1199723	0.0325	4107.2
tiktoken	technical	4.467	1044001	0.0355	3963.2
sentencepiece	code	4.154	1596882	0.0482	5499.2
sentencepiece	english	3.848	1491693	0.0431	4136.0
tiktoken	code	3.730	1121487	0.0539	5018.4
hf	english	3.713	628600	0.0446	7472.0
sentencepiece	technical	3.664	1404179	0.0431	4736.0
sentencepiece	web	3.392	1541396	0.0437	5064.0
hf	technical	2.974	709048	0.0531	7620.0

Domain Sensitivity: MB/s by Domain

Throughput shifts materially by domain, which changes the ranking of libraries.

domain	cached	hf	naive	sentencepiece	tiktoken
code	1.354	2.624	1.176	4.154	3.730
english	2.608	3.713	1.593	3.848	5.680
technical	2.282	2.974	1.353	3.664	4.467
web	1.759	2.968	1.246	3.392	4.562

Cold Start and Batch Results

Cold start is reported for all tokenizers. Batch metrics are available only for adapters with batch support.

tokenizer	metric	value	docs_per_s	tokens_per_s	batch_latency_ms
naive	cold_start_ms	0.0281			
tiktoken	cold_start_ms	0.0311			
cached	cold_start_ms	0.0356			
sentencepiece	cold_start_ms	0.0451			
hf	cold_start_ms	0.0918			
cached			63240.9	9393136	0.1485
naive			38256.1	5753732	0.2163
hf			28512.7	1150566	0.2943
sentencepiece			14607.6	991595	0.5543

Input-Length Scaling Summary

The current persisted corpus produced two dominant length buckets, so the length-scaling analysis is informative but not yet broad.

tokenizer	length_bucket	latency_ms	bytes	tokens
cached	257-1024	0.1967	308.0	248.0
cached	65-256	0.0858	170.1	145.5
hf	257-1024	0.1047	308.0	84.0
hf	65-256	0.0530	170.1	39.5
naive	257-1024	0.2356	308.0	248.0
naive	65-256	0.1220	170.1	145.5
sentencepiece	257-1024	0.0672	308.0	107.0
sentencepiece	65-256	0.0430	170.1	65.7
tiktoken	257-1024	0.0699	308.0	78.0
tiktoken	65-256	0.0356	170.1	39.3

Caching Optimization Results

Repeated-substring caching improved throughput and latency relative to the naive Python baseline, with a memory cost.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_byte	cache_hit_rate
cached	2.001	1794249	0.0927	8390.2	0.9947
naive	1.342	1201147	0.1291	6462.9	N/A

Results and Discussion

Headline findings

Overall winner by mean MB/s: tiktoken at 4.610 MB/s and 0.0378 ms average latency.

Fastest code-domain tokenizer: sentencepiece at 4.154 MB/s.

Cached versus naive Python baseline: 1.342 to 2.001 MB/s, or 1.49x speedup, with peak memory rising from 6462.9 to 8390.2 bytes.

Cold start ranking by mean latency favored naive (0.0281 ms) and tiktoken (0.0311 ms), while hf was slowest at 0.0918 ms.

Batch throughput favored the cached baseline at 63240.9 docs/s, but tiktoken is absent from the batch table because the current adapter intentionally exposes single-document encoding only.

Discussion

The results support H1. Native tokenizer libraries clearly outperformed the Python baselines on MB/s and latency, with tiktoken and SentencePiece consistently ahead of naive Python code.

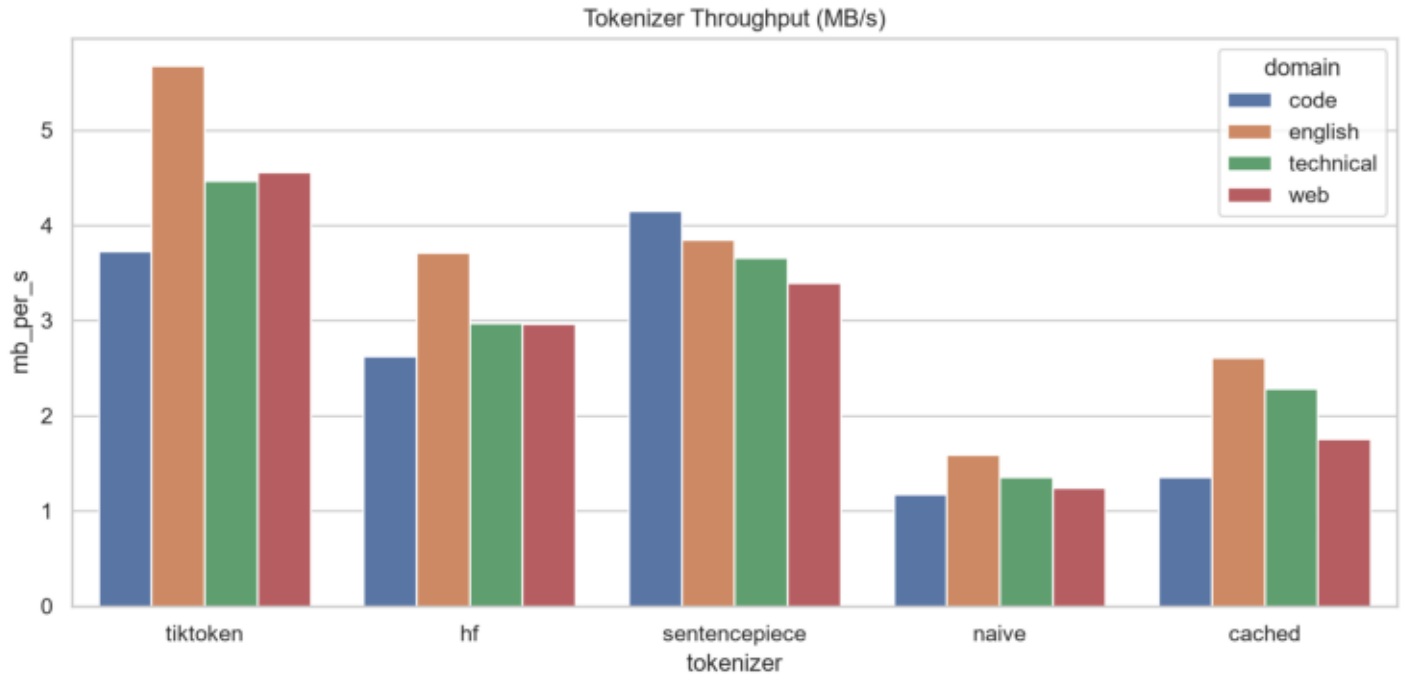
The results support H2. Ranking changed by domain: tiktoken led English, web, and technical text, while SentencePiece led code. That matters because tokenizer selection for RAG preprocessing, web corpora, and code corpora should not rely on one aggregate benchmark number.

The results support H3 with caveats. Caching materially improved the Python baseline, but it increased memory and still depended on high substring reuse. Even after diversifying the synthetic corpus, the cache hit rate remained high, so the next step is to rerun on real corpora.

The results partially support H4. Batch throughput was much higher for adapters with encode_batch support, but batch results are not comparable for tiktoken until a batch-capable adapter path is added.

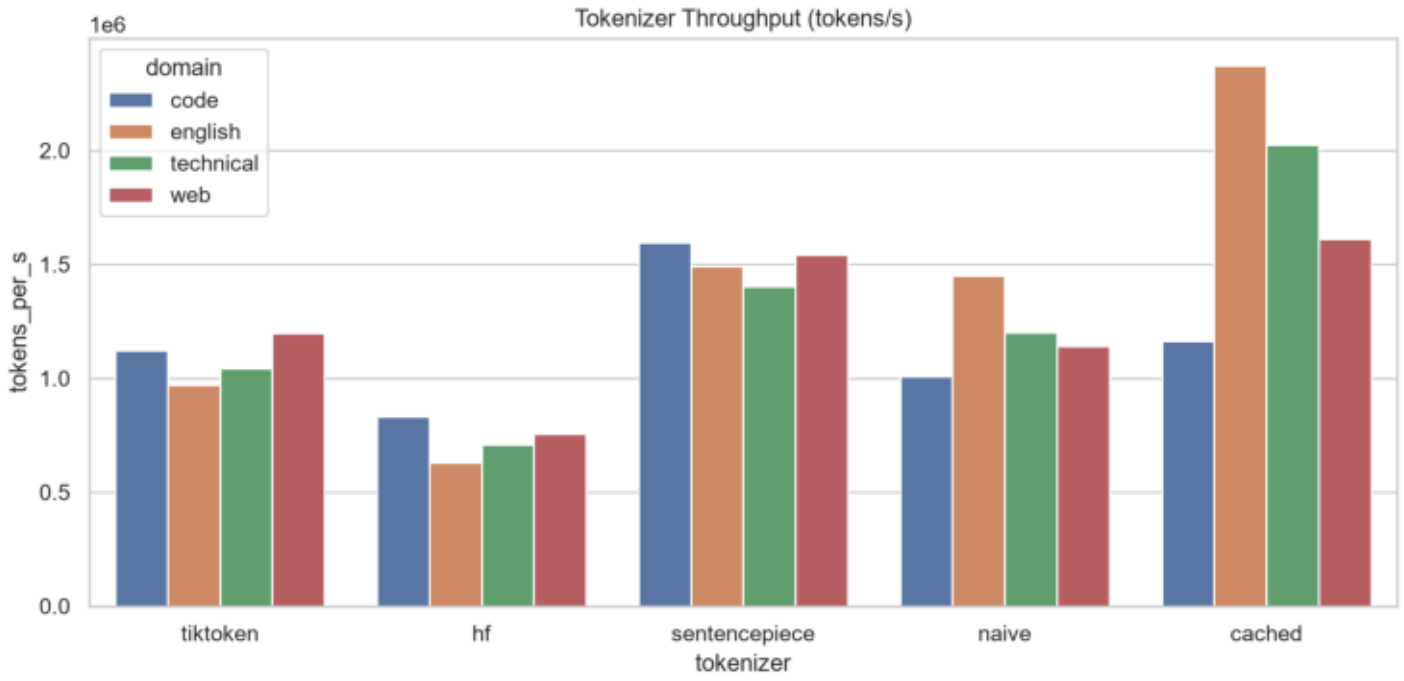
H5 remains unresolved empirically. The benchmark infrastructure is ready for exactness validation, but the project still needs a truly tiktoken-compatible optimized tokenizer before any claim about exact accelerated OpenAI-compatible BPE can be defended.

Throughput Mb S



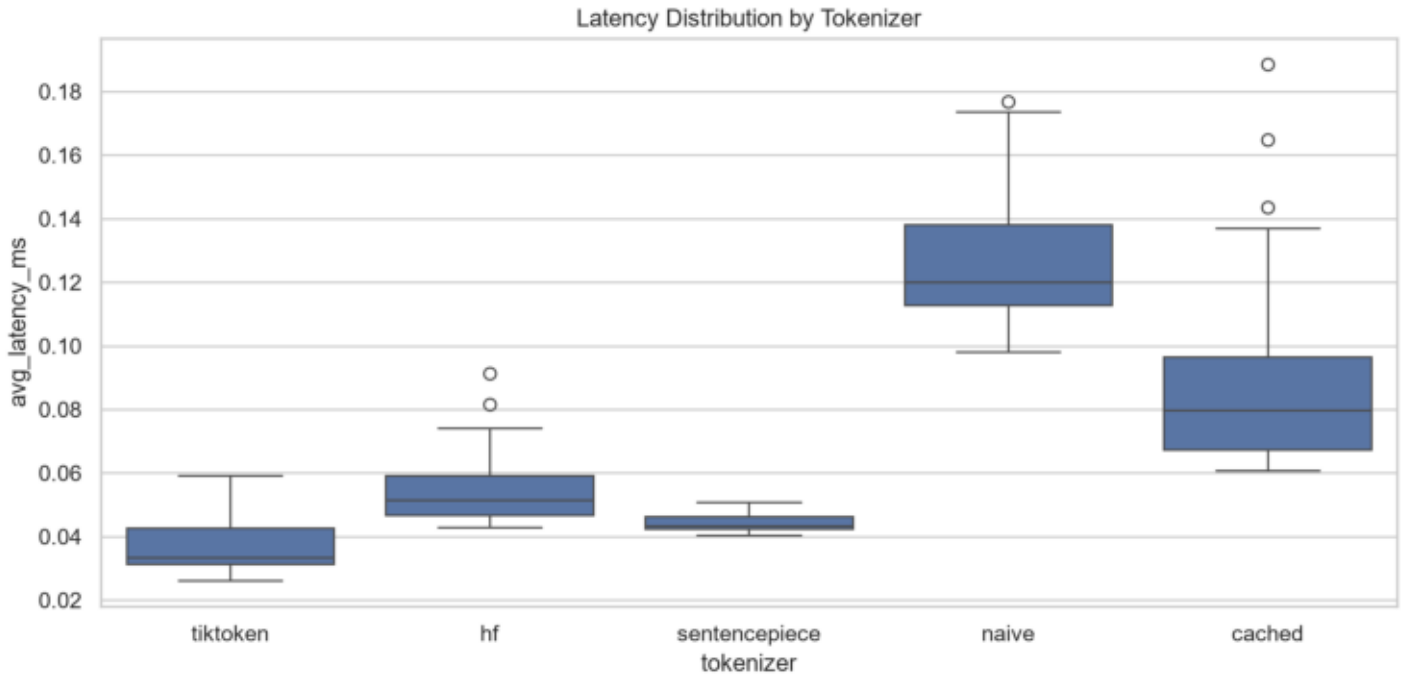
Experiment 1: Tokenizer throughput comparison in MB/s.

Throughput Tokens S



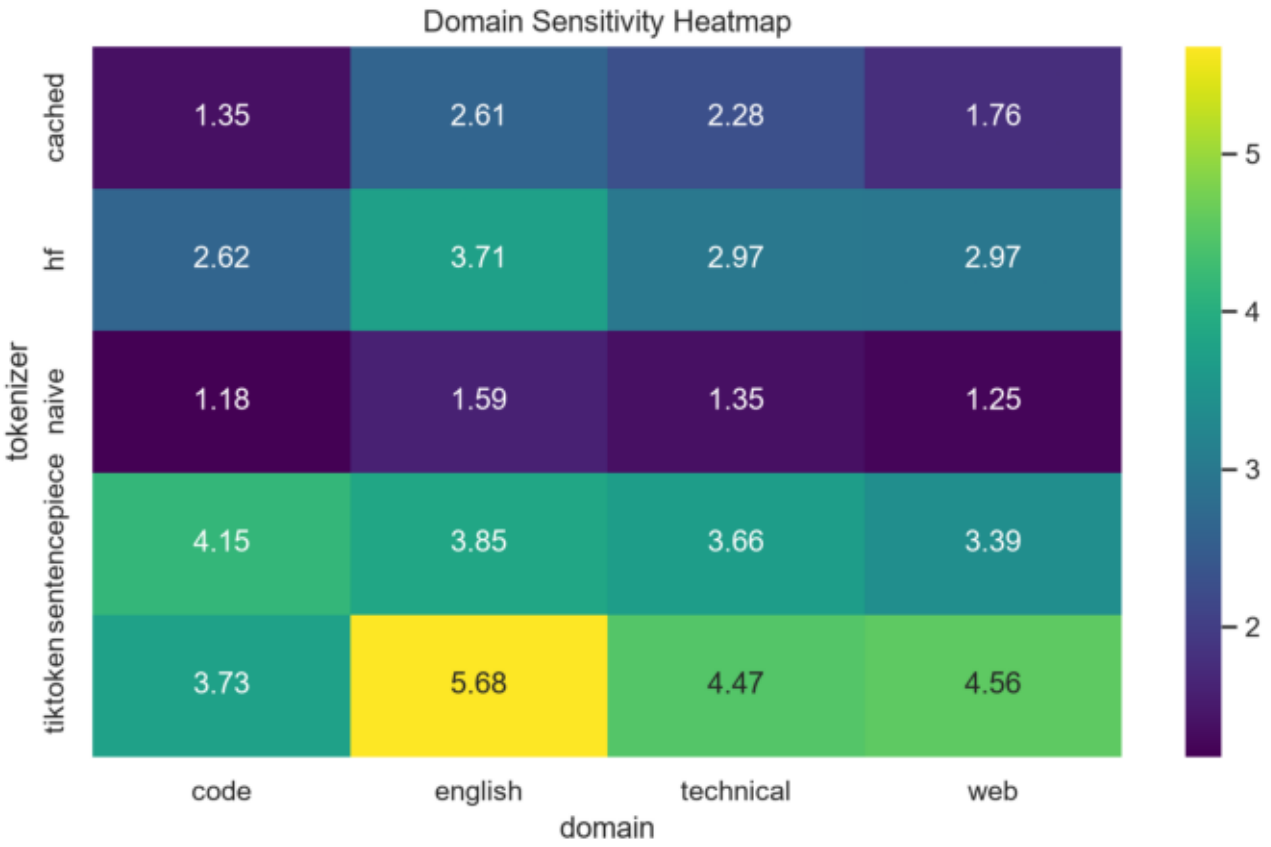
Experiment 1: Tokenizer throughput comparison in tokens/s.

Latency Distribution



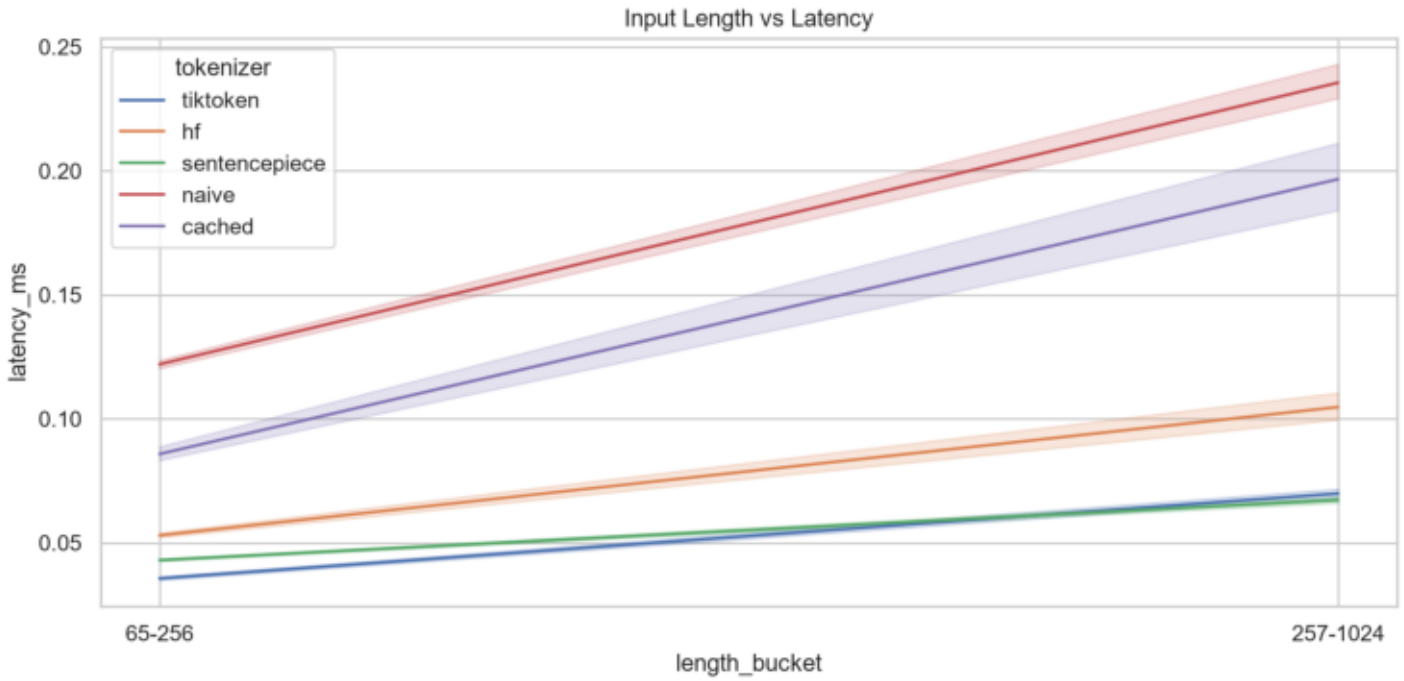
Latency distribution by tokenizer across benchmark trials.

Domain Heatmap



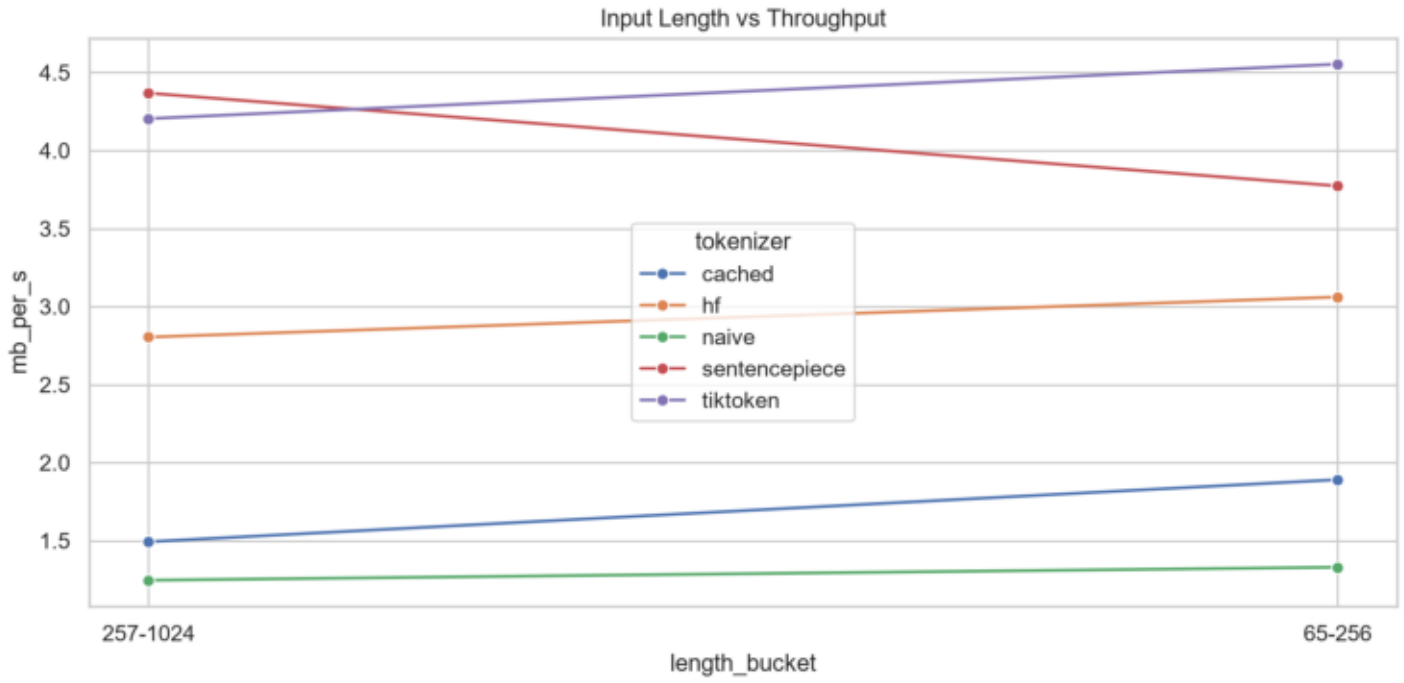
Experiment 2: domain sensitivity heatmap for throughput.

Length Vs Latency



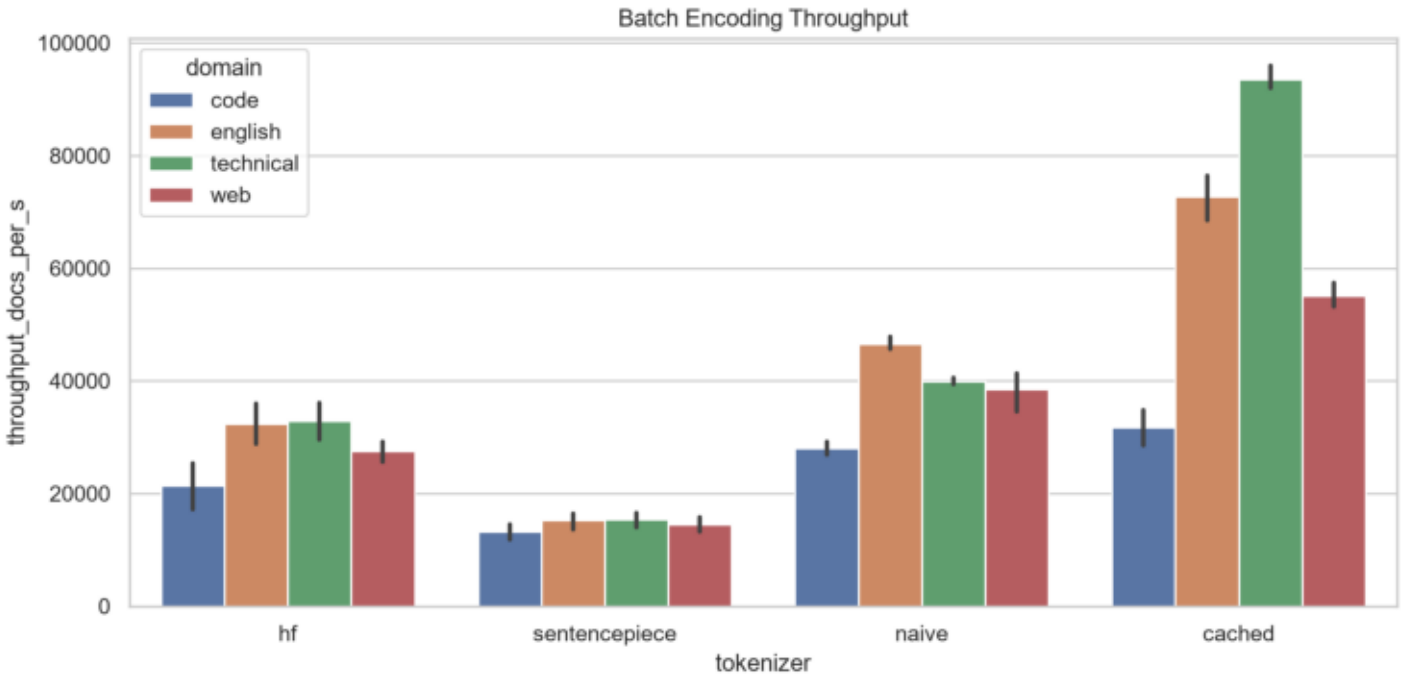
Experiment 3: input length versus latency.

Length Vs Throughput



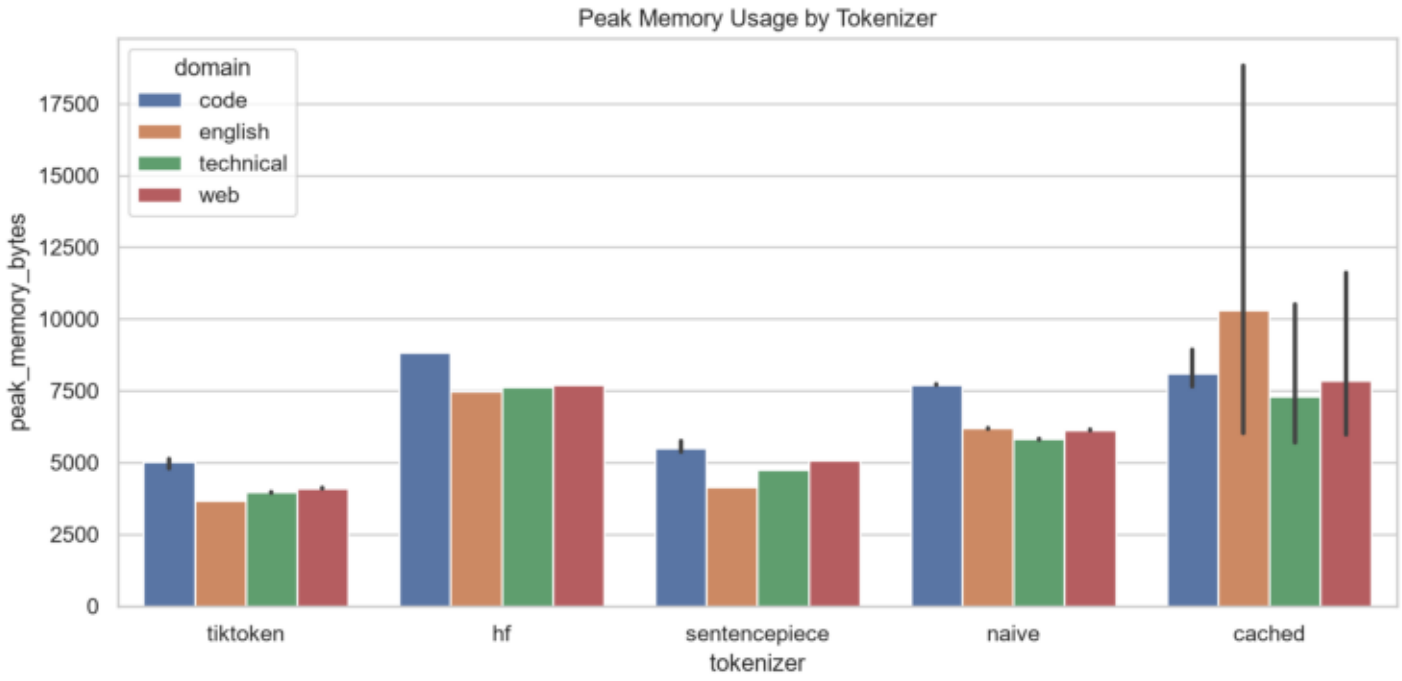
Experiment 3: input length versus throughput.

Batch Throughput



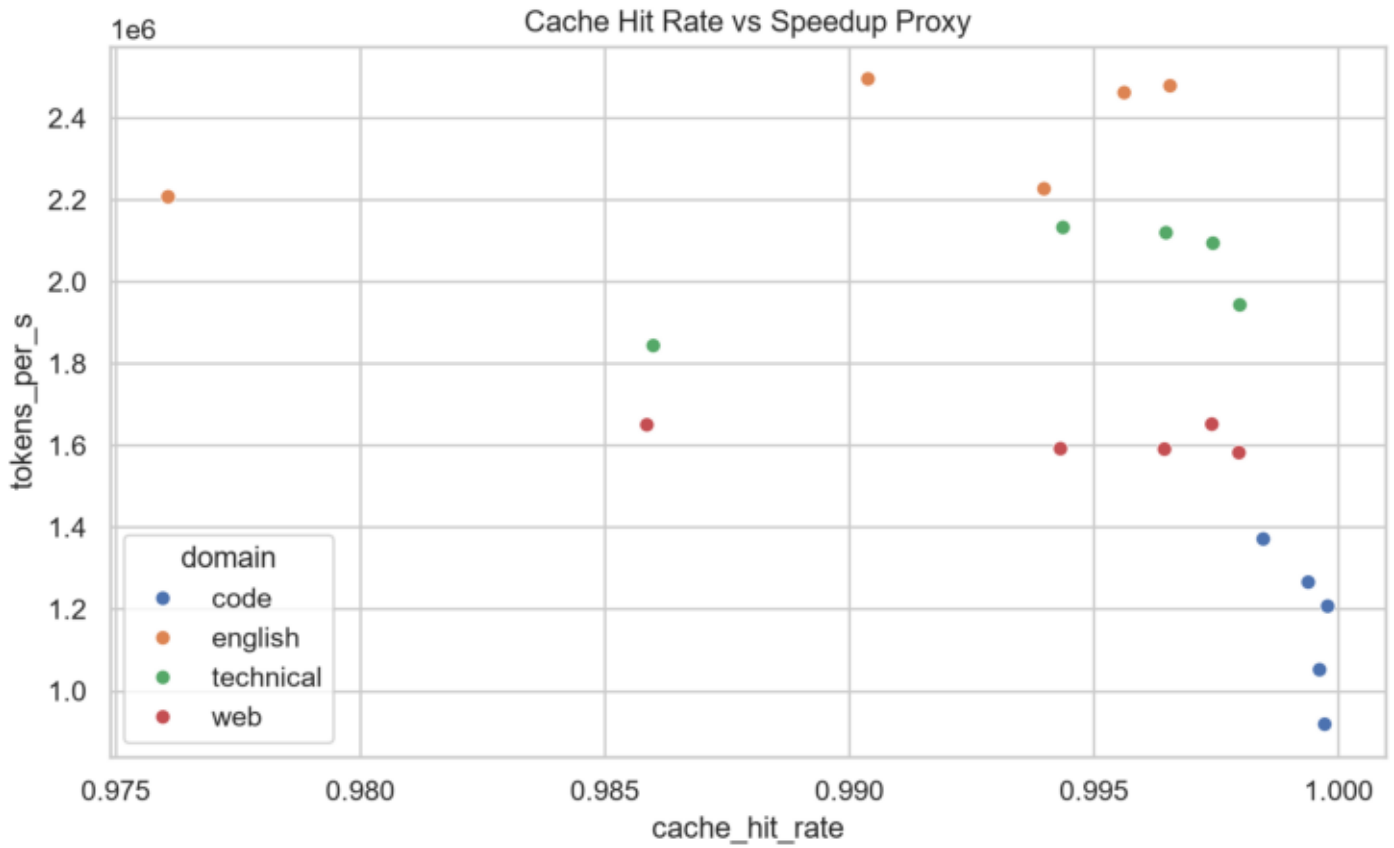
Experiment 4: batch encoding throughput.

Memory Usage



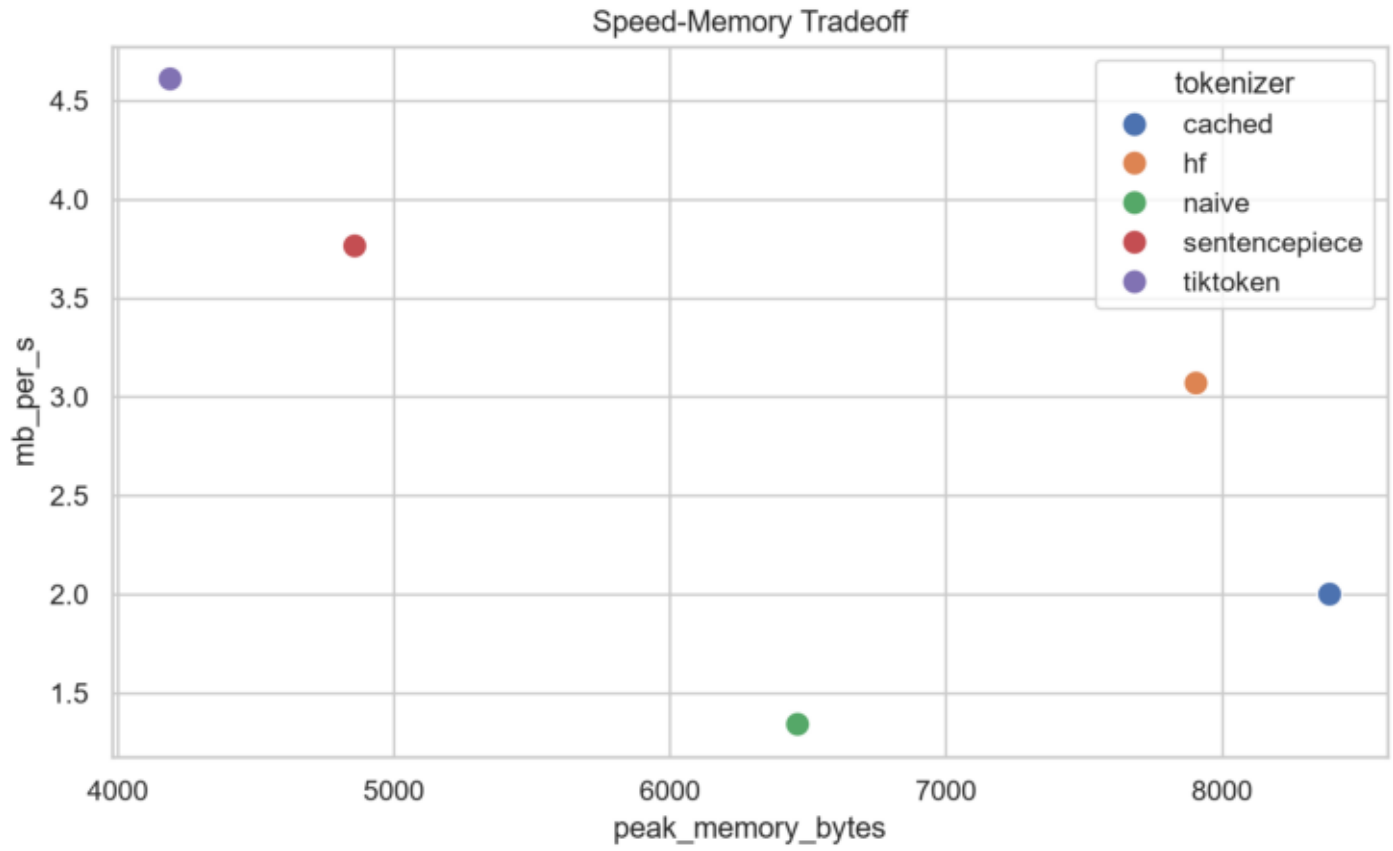
Memory usage by tokenizer and domain.

Cache Hit Rate Vs Speedup



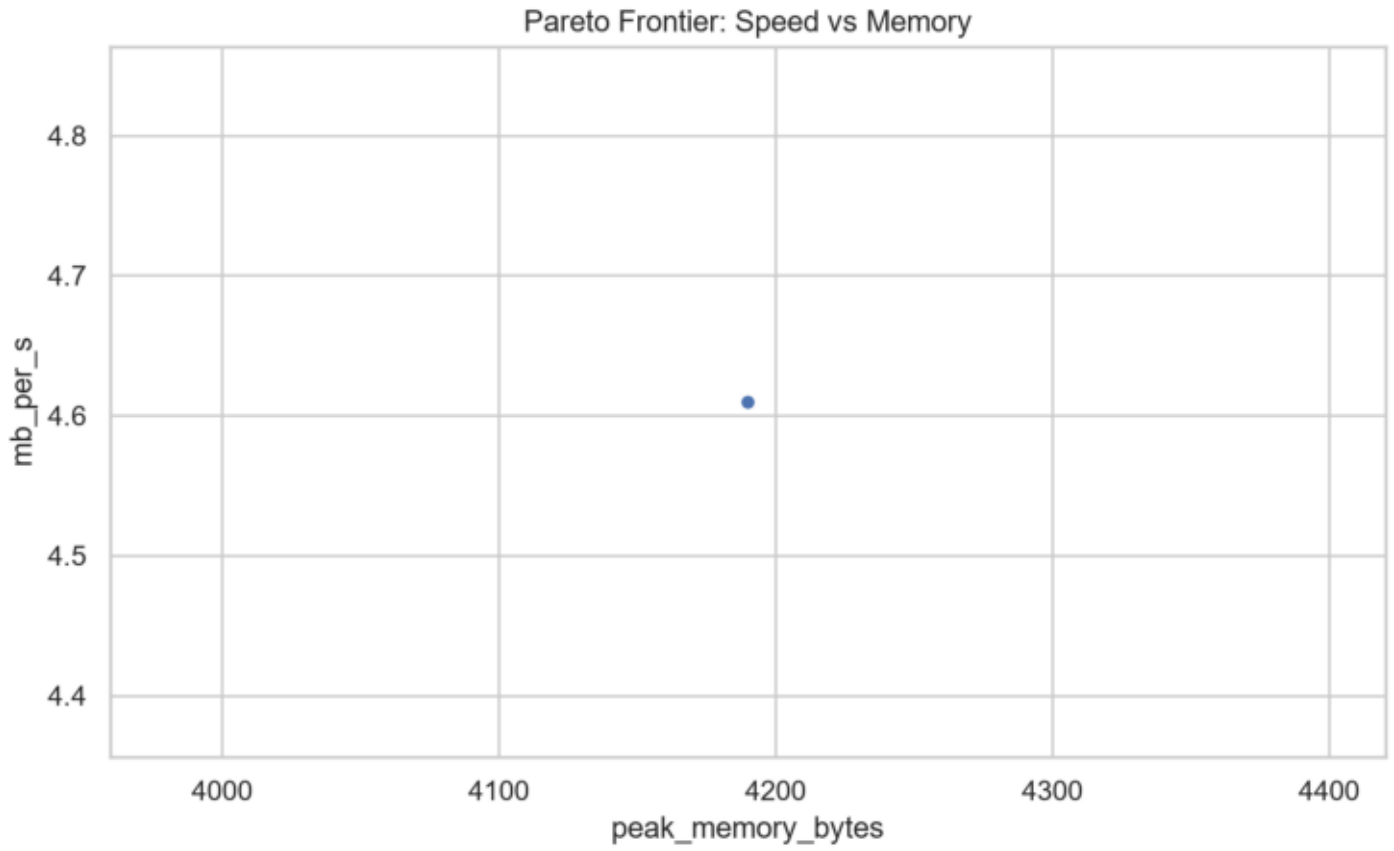
Experiment 6: cache hit rate versus throughput proxy.

Speed Memory Tradeoff



Speed-memory tradeoff across tokenizer implementations.

Pareto Frontier



Pareto frontier of throughput versus memory.